DOSSIER TECHNIQUE · SOUTENANCE

PROJET DE GESTION DOCUMENTAIRE AUGMENTÉE

DocRAG

Architecture, API, authentification, indexation sémantique, génération de réponses, interface Angular et exploitation.

FastAPI
API REST et orchestration

LanceDB
index vectoriel persistant

Angular
interface utilisateur

Ce dossier décrit le comportement observé dans le code du dépôt. Il donne les chemins de fichiers, les responsabilités, les fonctions et les données importantes afin de soutenir une présentation technique argumentée.

Périmètre
Backend Python · API REST · RAG · MySQL/LDAP · Frontend Angular · Docker/Kubernetes

Édition
3 octobre 2026
Document généré à partir du dépôt fourni

Sommaire

1. Résumé et périmètre fonctionnel
2. Architecture logique et flux de données
3. Structure du dépôt et frontières des composants
4. Pipeline documentaire et indexation
5. Chemin d'une question et analyse de query.py
6. Catalogue des API HTTP
7. Identité, autorisations et données SQL
8. Configuration, variables et valeurs par défaut
9. Application Angular : navigation, services et composants
10. Exploitation Docker et Kubernetes
11. Inventaire des fonctions et symboles du backend
12. Tests, validation et observabilité
13. Limites, risques et améliorations défendables
14. Trame de soutenance et questions techniques
15. Glossaire et repères de code

> **Lecture honnête du dépôt.** La stack principale décrite est la stack complète du Compose racine. Le dépôt contient aussi une CLI, une seconde implémentation RAG historique et un petit projet React de reporting; ces éléments ne sont pas sur le chemin des appels de l'interface Angular vers l'API.

1 · Résumé et périmètre fonctionnel

DocRAG est une application web de questions-réponses sur une collection de documents. Un administrateur dépose des fichiers, déclenche leur chargement, leur segmentation et leur indexation vectorielle. Un utilisateur authentifié pose une question; le serveur recherche quelques segments pertinents, les transmet à un modèle de langage exécuté via Ollama et retourne une réponse avec des références textuelles facultatives.

Objectifs visibles

- Centraliser des documents PDF, HTML et texte.
- Interroger leur contenu en langage naturel, en français.
- Réserver l'administration documentaire au rôle `admin`.
- Conserver séparément les documents, l'index vectoriel, les profils et les modèles LLM.

Rôles applicatifs

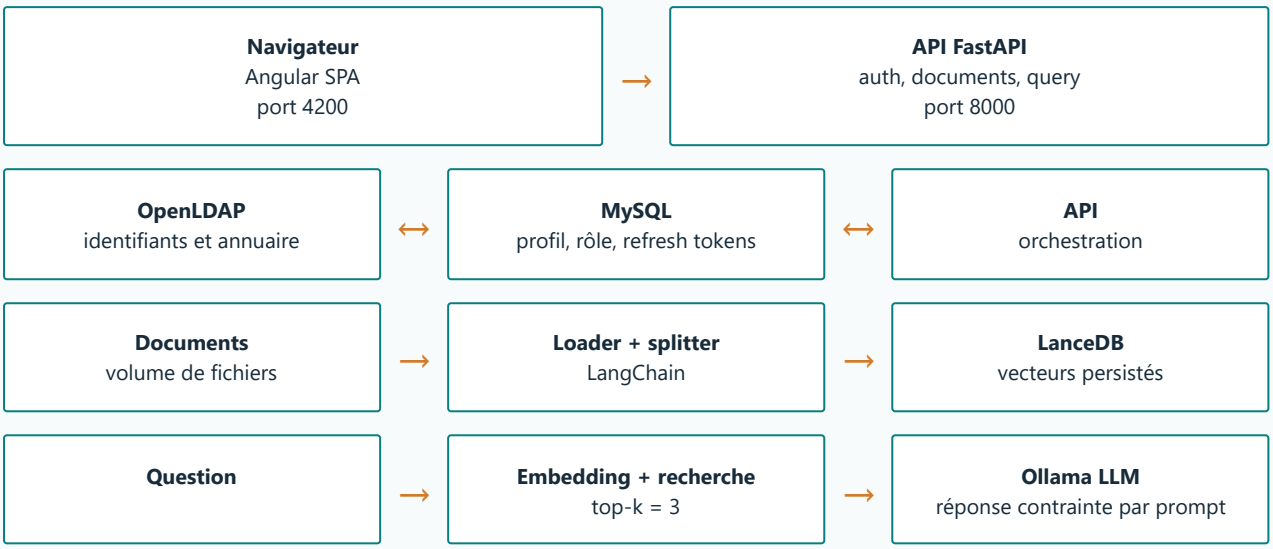
- **Visiteur :** connexion ou inscription.
- **Utilisateur :** accès au chat documentaire.
- **Administrateur :** chat, liste, upload, réindexation et suppression des fichiers.

Contrat de réponse RAG

La réponse principale est un texte (`answer`). Si le client demande les sources, le serveur fournit une liste contenant le chemin/nom de source et un extrait limité à 300 caractères. Ce sont des traces utiles pour l'utilisateur; ce mécanisme n'est pas une preuve formelle de vérité ni une citation de page PDF.

> **Positionnement pour la soutenance :** c'est un système RAG local/hybride, pas un modèle entraîné sur les fichiers. L'indexation prépare les représentations des documents; au moment de la requête, le système récupère du contexte et demande au LLM de formuler une réponse contrainte par ce contexte.

2 · Architecture logique



Responsabilités et frontières

Couche	Responsabilité	État persistant
Angular	Formulaires, navigation, état de session, chat et opérations admin.	Tokens de session dans `localStorage`.
FastAPI	Validation HTTP, dépendances d'auth, contrôle du rôle, coordination des services.	Instance RAG globale par processus; les documents ne sont pas stockés en mémoire uniquement.
LDAP	Bind du mot de passe et attributs d'annuaire.	Comptes et entrées LDAP.
MySQL	Profil applicatif, rôle, état actif, dates et refresh tokens hachés.	Tables `users`, `refresh_tokens`.
RAG	Chargement, découpe, embeddings, recherche de segments et génération.	Fichiers, table LanceDB, modèles Ollama et cache Hugging Face via volumes.

Pourquoi deux stockages pour l'authentification ?

LDAP est le référentiel des identifiants/mots de passe et attributs d'annuaire. MySQL porte les attributs propres à l'application, en particulier rôle et activation, ainsi que l'état de révocation des refresh tokens. Le login vérifie LDAP, puis synchronise ou retrouve l'utilisateur MySQL avant de délivrer les tokens.

3 · Structure du dépôt

Chemin	Rôle
`docker-compose.yml`	Orchestration principale complète : MySQL, OpenLDAP, seed, phpLDAPadmin, Ollama, API et frontend.
`sys_gestion_doc/api/`	Application FastAPI, routeurs, modèles Pydantic, auth LDAP/JWT, session et modèles SQLAlchemy.
`sys_gestion_doc/config.py`	Configuration partagée du RAG, des bases, de LDAP et des tokens.
`sys_gestion_doc/loader.py`, `vectorstore.py`, `embeddings.py`, `llm.py`, `rag.py`	Pipeline de préparation, stockage vectoriel et génération RAG utilisé par les routes.
`sys_gestion_doc/documents/`	Sources documentaires lues par l'indexeur; stockage persistant monté en conteneur.
`rag-frontend/src/app/`	Application Angular standalone : auth, chat, documents, guards, intercepteur et clients HTTP.
`rag-frontend/Dockerfile`, `nginx.conf`	Build Angular multistage et distribution statique avec fallback SPA.
`mysql/init/01-schema.sql`	Schéma initial du stockage applicatif.
`k8s/`	Namespace, secrets/configmaps, déploiements et services Kubernetes.
`scripts/`	Commandes PowerShell de démarrage, arrêt, seed et déploiement.
`reporting/digramme/`	Projet React distinct, initialisé Create React App; son README est le boilerplate par défaut, sans preuve d'intégration à l'interface Angular.

Chemins non utilisés par le serveur HTTP

`sys_gestion_doc/main.py` propose une interface terminal indépendante. `sys_gestion_doc/rag_system.py` contient une autre classe/pipeline autonome. `DbReaderApp.py`, `LanceDbReader.py` et `viewer.py` sont des outils de lecture/visualisation. Ils peuvent aider au diagnostic, mais les routes HTTP actives utilisent `api/main.py`, `rag.py` et `vectorstore.py`.

> **À ne pas confondre :** le Compose racine porte la stack complète. Le Compose situé dans le backend est une variante minimale/ancienne, et ne décrit pas à lui seul le déploiement principal.

4 · Pipeline documentaire et indexation

Étapes avant la première question

1. **Upload :** `POST /documents/upload` vérifie l'extension et écrit les octets dans `DOCUMENTS_FOLDER`. Il ne charge ni ne vectorise le fichier.
2. **Découverte :** `load_documents()` explore récursivement le dossier et ignore les extensions non supportées.
3. **Extraction :** PDF via `PyPDFLoader`; HTML via `UnstructuredHTMLLoader` (avec téléchargement de ressources NLTK); TXT/RAW via `TextLoader` UTF-8.
4. **Segmentation :** `RecursiveCharacterTextSplitter`, taille 1000 caractères, recouvrement 200, séparateurs préférés paragraphes puis lignes, espaces et caractères.
5. **Représentation :** `HuggingFaceEmbeddings`, modèle par défaut `sentence-transformers/all-MiniLM-L6-v2`, appareil CPU.
6. **Persistance :** création/remplacement de la table LanceDB `documents` dans `LANCE_DB_PATH`.
7. **Actualisation :** la route remplace le vectorstore en mémoire et appelle `reset_qa_chain()` pour reconstruire la chaîne à la prochaine question.

Pourquoi le recouvrement des chunks ?

Les frontières de segments peuvent couper une phrase ou une notion. Un recouvrement de 200 caractères conserve une partie du contexte local entre deux segments. Le compromis est une redondance accrue à l'indexation et potentiellement dans les résultats récupérés.

Pipeline à l'exécution

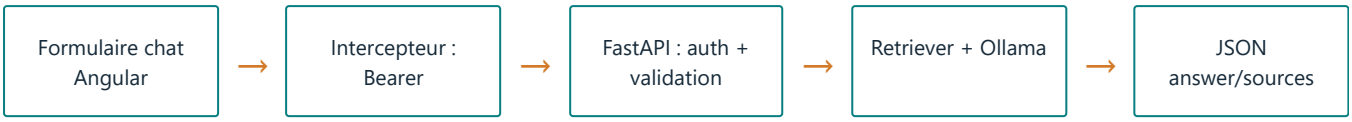
La question est transformée en embedding par le même modèle que celui utilisé pour les documents; le retriever LanceDB sélectionne au plus trois chunks selon la recherche configurée par l'intégration. La chaîne LangChain `RetrievalQA` de type `stuff` rassemble ces chunks dans le contexte du prompt et appelle le LLM Ollama. Le prompt demande une réponse courte en français, uniquement à partir du contexte, et une formule explicite si l'information manque.

> **Mesure importante :** `documents_indexed` est calculé par `len(documents)`, donc il représente le nombre d'objets `Document` chargés par les loaders, pas le nombre de fichiers, ni le nombre de chunks vectorisés. Un PDF multipage peut produire plusieurs objets.

Réindexation et cohérence

La réindexation est complète, pas incrémentale : la route supprime le dossier LanceDB avant de charger et reconstruire. L'index est donc recalculé à partir des fichiers actuellement présents. La suppression d'un fichier ne met pas à jour LanceDB immédiatement; il faut réindexer pour retirer son contenu de la recherche.

5 · Chemin d'une question et analyse de `query.py`



Le navigateur appelle `RagService.query(question, showSources)`. Le payload utilise les noms API snake_case : `question` et `show_sources`. L'intercepteur ajoute l'access token. FastAPI valide le corps avec `QueryRequest`, puis la dépendance `get_current_user` exige un utilisateur actif. La route interroge le RAG, adapte les documents sources au modèle de réponse et renvoie `QueryResponse`.

Lecture des éléments de la route

Symbole	Rôle exact dans la route
`router`	Routeur FastAPI préfixé par `/query`, groupé sous le tag OpenAPI « RAG ».
`query_documents`	Handler asynchrone du POST vide après le préfixe, soit `POST /query`, avec schéma de sortie `QueryResponse`.
`request`	Objet `QueryRequest`; `question` porte le texte, `show_sources` contrôle le renvoi des références et vaut vrai par défaut.
`_user`	Utilisateur fourni par injection de dépendance; le préfixe underscore marque que le corps n'a pas besoin de son contenu. L'injection protège la route.
`rag`	Instance globale obtenue via `get_rag()`, initialisée au démarrage par `set_rag()`.
`result`	Dictionnaire retourné par `rag.query()`, avec `result` textuel et `source_documents` produit par LangChain.
`sources`	Commence à `None`; devient la liste de modèles `SourceDocument` uniquement si l'utilisateur la demande et que LangChain a renvoyé les documents.
`doc`	Un document LangChain récupéré. La source vient de `chunk_document_source()`; le contenu est tronqué à 300 caractères.
`e`	Exception attrapée autour de l'appel RAG et convertie en HTTP 500 avec son texte dans `detail`.

Deux sens distincts de "show sources"

La route appelle `rag.query(request.question, show_sources=False)` pour éviter l'affichage console des sources par la logique interne. Cela n'empêche pas LangChain de les retourner : `return_source_documents=True`. La route décide séparément si elle expose ces sources au client web.

> **Point de discussion :** une exception inattendue est renvoyée comme détail HTTP 500; pour une mise en production, on préférerait un message client générique et un journal serveur protégé. Le service ne limite pas la longueur ou le débit des questions dans cette route.

6 · Catalogue des API HTTP

Préfixes directs : `/auth`, `/query`, `/documents`. La documentation OpenAPI est disponible dans l'application à `/docs` et `/redoc`. Les erreurs de validation de corps FastAPI donnent typiquement 422.

Méthode · chemin	Accès	Entrée	Sortie et règles
`GET /`	Public	Aucune	Message, chemin Swagger et version.
`GET /status`	Public	Aucune	`status`, `documents_loaded`, `vector_store_exists`, `total_documents`.
`POST /auth/login`	Public	`username`, `password`	Access JWT + refresh opaque; erreurs 401 identifiants invalides, 403 compte applicatif désactivé.
`POST /auth/register`	Public	Identifiant, mot de passe, nom affiché/email facultatifs	Utilisateur role=user, statut 201; erreurs LDAP 400, profil déjà créé 409.
`POST /auth/refresh`	Public, refresh dans corps	`refresh_token`	Rotation du refresh et nouveau access; 401 si expiré, révoqué ou inconnu.
`POST /auth/logout`	Public, refresh dans corps	`refresh_token`	Révocation si trouvé; statut 204, sans corps.
`GET /auth/me`	Bearer access	Aucune	Profil public applicatif : id, username, email, display_name, role.
`POST /query`	Bearer access	`question`, `show_sources` facultatif (défaut true)	Réponse et sources nullable; 503 si RAG/vectorstore absent, 500 si erreur RAG.
`GET /documents`	Admin	Aucune	Tableau filename/size/type des fichiers découverts récursivement.
`POST /documents/upload`	Admin	Multipart form-data, champ `file`	Message, nom et taille; seuls PDF, HTML, TXT, RAW acceptés; 400 extension refusée.
`POST /documents/index`	Admin	Aucune	Reconstruction complète, message et nombre d'objets chargés; 400 si rien à indexer; 503 si RAG absent.
`DELETE /documents/{filename}`	Admin	Nom de fichier dans le chemin	Message de confirmation; 404 si fichier absent. L'index doit être reconstruit séparément.

Modèles d'échange Pydantic

Modèle	Champs	Usage
`QueryRequest`	question: str; show_sources: bool = true	Entrée de recherche.
`SourceDocument`	source: str; content: str	Référence compacte dans la réponse.
`QueryResponse`	answer: str; sources: liste optionnelle	Sortie chat.

`StatusResponse`	status, documents_loaded, vector_store_exists, total_documents	État indicatif.
`DocumentInfo`	filename, size, type	Ligne de la liste de fichiers.
`LoginRequest`, `RegisterRequest`, `RefreshRequest`	Credentials; profil optionnel; refresh token	Entrées auth.
`TokenResponse`, `UserResponse`, `MessageResponse`	Tokens/durée; profil; message	Sorties auth et opérations.

> **Interprétation du statut :** `GET /status` renvoie actuellement `status="online"` de façon constante. Le total correspond aux fichiers du dossier; ce n'est pas le nombre de documents indexés. Il ne vérifie pas que MySQL, LDAP ou Ollama répondent.

7 · Identité, autorisations et données SQL

Connexion et jetons

1. `authenticate_ldap()` tente le bind LDAP de l'utilisateur, puis lit ses attributs via une connexion admin.
2. `sync_user_after_ldap_login()` retrouve/crée le profil MySQL, synchronise attributs et date de connexion; le rôle applicatif est conservé en base.
3. `create_access_token()` signe un JWT HS256 avec subject, id, rôle, type et expiration.
4. Un refresh token aléatoire opaque est donné au navigateur; seule son empreinte SHA-256 est enregistrée dans MySQL.
5. Au refresh, l'ancien jeton est révoqué, puis un nouveau couple est émis. À la déconnexion, le client efface immédiatement son état local et demande la révocation du refresh.

Contrôle des routes

`get_current_user()` extrait le Bearer, valide signature/type/expiration, relit le profil par identifiant et exige qu'il soit actif. `require_admin()` compare le rôle SQL à `admin`. L'API, et non uniquement le guard Angular, impose donc l'autorisation des routes d'administration.

Table	Colonnes utiles	Relations
`users`	id, ldap_uid unique, email, display_name, ldap_dn, role enum user/admin, is_active, last_login_at, created_at, updated_at	Un utilisateur possède plusieurs refresh tokens; suppression en cascade.
`refresh_tokens`	id, user_id, token_hash unique, expires_at, revoked_at nullable	Clé étrangère users.id avec suppression en cascade.

Frontières de confiance

Le JWT protège l'accès API mais le refresh est un bearer secret. LDAP et MySQL doivent rester sur des réseaux privés; les secrets de démonstration ne conviennent pas à la production. En configuration par défaut, LDAP SSL est désactivé, le secret JWT a une valeur de développement, et Compose contient des identifiants de démonstration.

> **Stockage navigateur :** l'application Angular garde les tokens en `localStorage`. Une faille XSS pourrait les lire. Une évolution possible est un cookie HttpOnly/Secure/SameSite associé à une stratégie CSRF adaptée, avec CSP et durcissement des en-têtes.

8 · Configuration et variables

`config.py` charge le fichier local `sys_gestion_doc/.env`, puis lit les variables du processus. Compose injecte notamment les adresses internes de services et les chemins de volumes. Les valeurs sensibles locales ne sont volontairement pas reproduites dans ce dossier.

Variable ou constante	Valeur par défaut observée / sens
`LLM_MODEL`, `LLM_BASE_URL`, `LLM_TEMPERATURE`	`mistral`; endpoint local Ollama 11434; température 0.1.
`EMBEDDING_MODEL`, `EMBEDDING_DEVICE`	`sentence-transformers/all-MiniLM-L6-v2`; `cpu`.
`LANCE_DB_PATH`, `LANCE_TABLE_NAME`	`./lance_db`; `documents`.
`DOCUMENTS_FOLDER`, `SUPPORTED_EXTENSIONS`	`./documents`; .pdf/.html/.txt/.raw.
`TOP_K_CHUNKS`, `CHUNK_SIZE`, `CHUNK_OVERLAP`	3 chunks; 1000 caractères; recouvrement 200.
`PROMPT_TEMPLATE`	Français, réponse courte, sources de contexte uniquement, phrase de repli prédéfinie.
`DATABASE_URL`	URL SQLAlchemy MySQL/PyMySQL; Compose remplace l'hôte local par le service MySQL.
`JWT_SECRET`, `JWT_ALGORITHM`	Secret de développement par défaut; HS256.
`JWT_ACCESS_EXPIRE_MINUTES`, `JWT_REFRESH_EXPIRE_DAYS`	60 minutes; 7 jours.
`LDAP_SERVER`, `LDAP_PORT`, `LDAP_USE_SSL`	ldap://localhost; port 389; SSL false.
`LDAP_BASE_DN`, `LDAP_BIND_DN`, `LDAP_BIND_PASSWORD`	Base DN et compte admin de démonstration; à fournir/rotater hors développement.
`LDAP_USER_SEARCH_BASE`, `LDAP_USER_SEARCH_FILTER`, `LDAP_ORGANISATION`	OU users; filtre uid; organisation SysGesDoc.
`AUTH_DISABLED`	false par défaut; si vrai, la dépendance fournit un admin synthétique. À proscrire en production.
`SEED_ADMIN_USERNAME`, `SEED_ADMIN_DISPLAY_NAME`	Sahbi et Sahbi Admin dans les valeurs initiales.
`CORS_ORIGINS`	Lu dans `api/main.py`; liste séparée par virgules des origines frontend autorisées.
`API_URL`	Argument de build Docker Angular; remplace l'URL API dans les fichiers d'environnement.
`MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`, paramètres OpenLDAP	Variables Compose de conteneur, distinctes du lecteur `config.py`; les exemples sont destinés au développement.

Variables internes notables

Dans `api/main.py`, `_cors_env` contient la chaîne CORS brute et `CORS_ORIGINS` sa forme de liste. Pendant le startup, `db`, `embeddings`, `llm` et `rag` sont les ressources initialisées; les exceptions MySQL et le chargement de LanceDB sont traités séparément.

9 · Application Angular

Composition et navigation

Le client est une application Angular standalone. `main.ts` démarre le composant racine; `app.config.ts` fournit le routeur, HttpClient avec intercepteur, détection de changements et initialiseur de session. Les routes lazy-loadent les composants.

Route	Guard	Écran
`/login`	guestGuard	Connexion
`/register`	guestGuard	Inscription
`/chat`	authGuard	Conversation RAG
`/documents`	adminGuard	Gestion documentaire
`/` et joker	Redirection	Chat

Session et réseau

- `AuthService` expose signal utilisateur, `isAuthenticated`, `isAdmin`; restaure la session à l'initialisation et demande `/auth/me`.
- Les clés `rag_access_token` et `rag_refresh_token` sont stockées en `localStorage`.
- `authInterceptor` omet l'en-tête pour login/register/refresh/logout, ajoute Bearer aux autres appels et, sur 401, tente un refresh partagé puis rejoue la requête.
- `RagService` centralise les appels HTTP pour status, query et CRUD documentaire; il transforme les erreurs API en erreurs RxJS.
- `environment.apiUrl` vaut `http://localhost:8000` en développement; le Dockerfile reçoit une URL de build.

Composants et fonctions

Composant / service	État principal	Méthodes et rôle
App	status; AuthService	Effet réactif : charge status quand connecté; `logout()`; `displayName()`; `loadStatus()`.
`LoginComponent`	username, password, error, loading	`submit()` valide les champs, appelle login puis navigue chat ou affiche erreur.
`RegisterComponent`	username, password, displayName, email, error, success, loading	`submit()` valide présence et longueur minimale client de 8, construit le payload optionnel, affiche succès puis redirige vers login.
`ChatComponent`	messages, input, showSources, isLoading, placeholder DOM	`send()`; `onKeydown()`; `sendSuggestion()`; `clear()`; `scrollToBottom()`; `formatTime()`; `shortSource()`.
`DocumentsComponent`	documents, états chargement/indexation/upload/glisser-déposer, toast	`loadDocuments()`; `onFileSelected()`; `onDrop()`; `onDragOver()`; `onDragLeave()`; `uploadFiles()`; `indexDocuments()`; `deleteDocument()`; `formatSize()`; `typeIcon()`; `showToast()`.

`AuthService`	signal User|null; refreshInFlight	`bootstrap()`; `login()`; `register()`; `me()`; `logout()`; `clearSession()`; `refreshAccessToken()`; getters accessToken/refreshToken; `storeTokens()`; `handleError()`.
`RagService`	URL API	`getStatus()`; `query()`; `listDocuments()`; `uploadDocument()`; `indexDocuments()`; `deleteDocument()`; `handleError()`.

Interfaces TypeScript et UX de flux

`auth.models.ts` définit LoginRequest, RegisterRequest, TokenResponse, User et RefreshRequest. Le service partagé définit QueryRequest, SourceDocument, QueryResponse, StatusResponse, DocumentInfo et MessageResponse. Le chat affiche un placeholder d'assistant pendant l'appel, bloque un nouvel envoi tant que le précédent est actif, accepte Entrée (Maj+Entrée conserve le retour à la ligne), peut montrer les sources et gère les erreurs. La page documents accepte sélection et glisser-déposer, upload plusieurs fichiers, demande explicitement la réindexation et confirme avant suppression.

> **Détail à expliquer :** les guards améliorent l'expérience de navigation, mais ne remplacent pas `Depends(require_admin)` côté API. Le contrôle serveur demeure l'autorité.

10 · Exploitation Docker et Kubernetes

Docker Compose (chemin recommandé du dépôt)

Le Compose racine démarre MySQL 8, OpenLDAP, phpLDAPadmin, un job LDAP seed, Ollama, FastAPI et Nginx/Angular. Le réseau privé Compose permet aux conteneurs de s'appeler par leur nom de service. Les ports exposés usuels sont 3306, 389, 8081, 11434, 8000 et 4200.

Volume logique	Donnée conservée
`mysql_data`	Comptes applicatifs et refresh tokens.
`ldap_data`, `ldap_config`	Annuaire et configuration LDAP.
`lance_db`	Index vectoriel et table LanceDB.
`documents`	Fichiers envoyés au système.
`hf_cache`	Cache du modèle d'embeddings.
`ollama_data`	Modèles de langage téléchargés.

Le modèle Ollama doit être tiré avant utilisation effective (script `scripts/pull-model.ps1` ou commande Ollama dans le conteneur). Le healthcheck API s'appuie sur `/status`; comme cet endpoint ne vérifie pas ses dépendances, le résultat est plus faible qu'un vrai readiness check de tous les services.

Kubernetes

Les manifests sont regroupés sous le namespace `docrag` avec déploiements et services MySQL, LDAP, Ollama, rag-api et rag-frontend, ConfigMaps/secrets et job seed LDAP. Les Services sont internes de type ClusterIP; aucun Ingress n'est défini. L'accès documenté passe par `kubectl port-forward`. Les PVC indiqués sont 2 GiB pour MySQL, 1 GiB LDAP, 10 GiB Ollama et 5 GiB données API.

> **URL API Angular en cluster :** le Dockerfile prend `API_URL` comme argument. Le Compose le passe avec l'URL navigateur localhost. Le script Kubernetes décrit dans le dépôt construit les images sans cet argument; la valeur par défaut reste localhost:8000, d'où la nécessité du port-forward documenté. Pour un déploiement partagé, il faut injecter une URL joignable par le navigateur ou mettre en place une entrée HTTP appropriée.

Commandes de démonstration

```
docker compose up --build -d
.scripts\pull-model.ps1
# Vérifier l'API et sa documentation
http://localhost:8000/status
http://localhost:8000/docs
# Interface utilisateur
http://localhost:4200
```

11 · Inventaire des fonctions et symboles backend

Inventaire des fonctions de service exposées par les modules qui composent les parcours applicatifs. Les variables de configuration globales sont détaillées au chapitre 8; les attributs de composants Angular sont détaillés au chapitre 9.

Module / symboles	Responsabilité technique
`api/main.py` `app`, `_cors_env`, `CORS_ORIGINS`, `startup_event()`	Construit FastAPI, ajoute CORS, enregistre routeurs; au startup seed MySQL, crée le dossier, initialise embeddings/LLM/RAG, restaure LanceDB si présent et publie l'instance. Le bloc principal lance Uvicorn sur 0.0.0.0:8000.
`api/dependencies.py` `rag_system`, `get_rag()`, `set_rag(instance)`	Stocke et expose l'instance RAG globale au processus.
`api/routes/general.py` `root()`, `get_status()`	Retourne la bannière API et un état sommaire des fichiers/index et de l'instance.
`api/routes/query.py` `query_documents(request, _user)`	Protège la requête, vérifie l'index, interroge le RAG, forme la réponse et mappe les sources. HTTP 503 si non initialisé; erreurs internes HTTP 500.

`api/routes/documents.py`
`list_documents()`
`upload_document(file)`
`index_documents()`
`delete_document(filename)`
	Liste récursive; valide et persiste le fichier uploadé; détruit/reconstruit la base; retire un fichier du disque. Tous les handlers exigent admin.

`api/auth/router.py`
`_build_token_response(db,user)`
`login(body,db)`
`register(body,db)`
`refresh_token(body,db)`
`logout(body,db)`
`me(user)`
	Centralise les endpoints de session. Le helper émet access + refresh et persiste le hash du refresh; login vérifie LDAP, register crée LDAP puis SQL, refresh révoque/renouvelle, logout révoque, me sérialise le profil.

`api/auth/dependencies.py`
`_bearer`
`get_current_user(credentials,db)`
`require_admin(user)`
	Extrait Bearer, valide JWT, recharge le profil/état actif; mode AUTH_DISABLED fournit un admin de test; vérifie ensuite le rôle admin.

`api/auth/security.py`
`create_access_token()`
`decode_access_token()`
`generate_refresh_token()`
`hash_refresh_token()`
`refresh_token_expires_at()`
	Signature/décodage JWT, génération aléatoire opaque, hachage SHA-256 et calcul d'expiration UTC.

`api/auth/user_service.py`
`get_user_by_username()`
`ensure_admin_seed()`
`sync_user_after_ldap_login()`
`create_user_from_registration()`
`store_refresh_token()`
`revoke_refresh_token()`
`get_valid_refresh_token()`
	Opérations métier SQL pour trouver/synchroniser/amorcer utilisateurs et gérer cycle de vie des refresh tokens.

`api/auth/ldap_service.py`
`LDAPError`
`_server()`
`_admin_connection()`
`_user_dn()`
`authenticate_ldap()`
`register_ldap_user()`
`ensure_users_ou()`
	Construit les connexions LDAP, le DN utilisateur, vérifie credentials et attributs, crée un utilisateur, ou prépare l'OU users.

`api/db/session.py` `engine`, `SessionLocal`, `Base`, `get_db()`	Configure SQLAlchemy/MySQL, classe de base déclarative, sessions par requête avec fermeture garantie.
`api/db/models.py` `User`, `RefreshToken`	Modèles relationnels et relation un-à-plusieurs; cascade de suppression.
`api/models.py`	Schémas Pydantic cités au chapitre 6; validation et forme JSON des échanges.
`config.py`	Charge .env, fige constantes par processus, définit prompt, chemins, limites, secrets, délais et paramètres LDAP.
`loader.py` `load_documents(documents_path)`	Découvre les extensions supportées, choisit le loader et accumule les documents extraits; intercepte les erreurs par fichier.
`embeddings.py` `get_embeddings()`	Instancie l'encodeur Hugging Face avec modèle et device configurés.
`llm.py` `get_llm(model_name)`	Configure le wrapper LangChain Ollama : modèle, base URL, température.

`vectorstore.py`
`vector_index_exists()`
`build_vectorstore(documents,embeddings)`
`load_vectorstore(embeddings)`
	Vérifie table LanceDB, découpe récursivement les documents puis remplace la table; ou reconnecte la table existante.

`rag.py`
`chunk_document_source(doc)`
`RAGSystem.__init__()`
`_build_qa_chain()`
`qa_chain`
`reset_qa_chain()`
`query(question,show_sources)`
	Retrouve source dans metadata racine ou imbriquée; conserve vectorstore/LLM; construit paresseusement la chaîne QA, la réinitialise après indexation, exécute invoke et peut imprimer les sources.

État global et cycle de vie

L'objet RAG est conservé dans une variable de module; le démarrage le configure une fois par worker/processus. En cas de déploiement multi-worker, chaque processus a son propre objet en mémoire, bien que les données LanceDB soient partagées sur disque. Les modifications concurrentes de l'index ne sont pas coordonnées par un verrou distribué visible dans le dépôt.

12 · Tests, validation et observabilité

Tests backend

`pytest.ini` découvre `tests/test_*.py`, classes `Test*` et fonctions `test_*`. Les marqueurs définis sont unit, integration, slow et api. Les suites présentes couvrent configuration, embeddings, loaders, CLI, RAG, seconde implémentation RAG, vectorstore et API via TestClient/mocks. Les tests API isolent normalement les dépendances lourdes avec des mocks et le mode auth de test.

Tests Angular

Le service HTTP RAG a un fichier de tests Vitest dédié, couvrant les appels HTTP (unité rapportée dans l'inventaire du dépôt : huit cas). Les scripts npm proposent `npm test` et `npm run test:unit`. Un build production est prévu par `npm run build`.

Observabilité disponible

- Messages de démarrage et erreurs imprimés par l'API et les loaders.
- OpenAPI interactif dans Swagger/ReDoc.
- Logs Compose par service.
- Healthcheck conteneur API basé sur `/status`, frontend Nginx basé sur HTTP local.

> **Transparence :** les tests n'ont pas été exécutés pour la création de ce dossier. Il décrit les fichiers de tests présents et ne revendique pas un statut de réussite. Avant la soutenance, lancer les suites dans les environnements réellement installés et noter les résultats.

Scénario de validation manuel efficace

1. Vérifier que les services sont healthy et qu'Ollama contient le modèle requis.
2. Créer/se connecter avec un utilisateur; vérifier `GET /auth/me`.
3. Vérifier qu'un utilisateur standard ne peut pas ouvrir les routes admin (403 API).
4. En admin, charger un TXT court, confirmer qu'il apparaît dans la liste puis déclencher l'indexation.
5. Poser une question dont la réponse figure dans le fichier, puis une question absente du contexte.
6. Activer les sources et vérifier nom/extrait; supprimer le fichier, réindexer et contrôler qu'il ne ressort plus.
7. Tester une requête sans access token et l'expiration/rotation du refresh token.

13 · Limites et pistes d'amélioration

Constat observé	Conséquence	Amélioration défendable
`/status` n'interroge pas réellement les dépendances et annonce toujours online.	Un healthcheck peut être vert alors que le service LLM ou SQL est indisponible.	Séparer liveness/readiness; tester dépendances essentielles et états distincts.
Le total status compte les fichiers disque, pas les objets ou chunks indexés.	Le libellé UI peut surinterpréter cette métrique.	Publier des compteurs distincts : fichiers, documents chargés, chunks/table.
Suppression fichier sans mutation immédiate de LanceDB.	Contenu supprimé encore récupérable jusqu'à réindexation.	Réindexation transactionnelle ou suppression ciblée par source; confirmer succès avant remplacer index.
Reconstruction supprime la base avant d'avoir fini le chargement/création.	Échec en cours peut laisser l'index absent ou incomplet.	Construire dans un emplacement temporaire, valider, puis effectuer un échange atomique.
Upload sans limite de taille/nom normalisé visible dans ce handler.	Risque de saturation ou de chemins inattendus selon noms reçus.	Bornes de taille, nom serveur généré, vérification résolution de chemin, quotas et antivirus selon menace.
Refresh token en localStorage; secret JWT et valeurs LDAP de dev.	Vol possible via XSS; configuration impropre à la production.	Secrets externes rotatés, TLS, cookie HttpOnly, CSP, contrôle des accès et politique de rotation.
Erreur RAG brute incluse au détail 500.	Peut divulguer détails d'infrastructure.	Message client stable, logs internes corrélés sans secrets.
Prompt réduit les réponses hors contexte mais ne supprime pas l'injection de prompt documentaire.	Des documents peuvent contenir des instructions adverses ou du contenu trompeur.	Traiter les documents comme données non fiables, filtrer/évaluer les sorties, limiter outils et permissions du LLM.
Pas de pagination ni filtres dans liste fichiers; sources sans page PDF.	Ergonomie/traçabilité limitées avec grandes collections.	Pagination et métadonnées d'origine (page, source stable), provenance affichée.

Ces points ne sont pas une critique abstraite : chacun est relié à une propriété précise des handlers ou des valeurs de configuration. En soutenance, les présenter comme des choix/limites connus puis prioriser les améliorations renforce la crédibilité technique.

14 · Trame de soutenance

Présentation orale en 90 secondes

> « DocRAG est une application de gestion documentaire avec recherche augmentée par génération. L'administrateur dépose des documents et déclenche leur indexation. Le backend extrait le texte, le segmente avec recouvrement, calcule des embeddings et les persiste dans LanceDB. Lorsqu'un utilisateur pose une question, le retriever cherche les trois segments les plus pertinents et LangChain les injecte dans un prompt destiné à Ollama. La réponse peut inclure les sources récupérées. L'API FastAPI porte la validation et les contrôles d'accès; LDAP authentifie, tandis que MySQL gère le profil applicatif, le rôle et les refresh tokens. Angular fournit le chat et l'administration. Docker Compose orchestre la stack; des manifests Kubernetes existent pour un déploiement de démonstration. »

Déroulé de démonstration

1. Montrer le schéma et distinguer stockage des fichiers et index vectoriel.
2. Se connecter comme admin et expliquer LDAP/MySQL sans révéler de secrets.
3. Uploader un document : insister sur le fait que l'upload n'indexe pas.
4. Déclencher indexation : expliquer extraction, chunks 1000/200, embedding et LanceDB.
5. Poser une question avec réponse présente puis une question hors corpus.
6. Afficher sources et ouvrir Swagger pour relier l'écran à `POST /query`.
7. Expliquer la rotation refresh et la protection serveur des endpoints admin.
8. Terminer par deux limites maîtrisées : métrique status approximative et index à reconstruire après suppression.

Questions probables et réponses courtes

Question	Réponse techniquement juste
Pourquoi RAG plutôt que fine-tuning ?	Le corpus peut être modifié et réindexé sans réentraîner le modèle; le système fournit un contexte récupéré et des références de source. Il faut néanmoins évaluer la qualité de retrieval et génération.
À quoi sert un embedding ?	À représenter sémantiquement un texte par un vecteur pour comparer une question à des segments documentaires dans l'index.
Pourquoi 3 chunks ?	C'est le réglage top-k actuel, un compromis entre contexte utile, coût et bruit; il devrait être calibré par évaluation sur le corpus.
Que se passe-t-il si Ollama est arrêté ?	L'API peut démarrer, mais la requête RAG échouera au moment de l'appel au modèle; le status actuel ne garantit pas la disponibilité d'Ollama.
Qui décide qu'un utilisateur est admin ?	Le rôle du profil SQL relu à partir du subject du JWT, vérifié dans la dépendance serveur `require_admin`; pas le seul guard Angular.
Pourquoi refresh opaque en plus d'un JWT ?	Le JWT access est stateless et de courte durée; le refresh conservé côté serveur sous hash permet expiration, révocation et rotation.
Les fichiers uploadés sont-ils immédiatement interrogeables ?	Non. L'upload écrit le fichier; il faut lancer explicitement la réindexation.
Que représente le nombre « indexed » ?	La route compte les objets LangChain chargés, non les fichiers ni les chunks. La mesure mérite un libellé plus précis.

Comment éviter une réponse inventée ?	Le prompt contraint le modèle au contexte et prévoit une réponse d'absence. Cette consigne réduit le risque, mais ne garantit pas une absence totale d'hallucination; tests et évaluation restent nécessaires.

15 · Glossaire et repères de code

Terme	Définition dans ce projet
RAG	Retrieval-Augmented Generation : récupérer du contexte depuis le corpus avant de générer.
Chunk	Segment de texte découpé depuis un document et traité comme unité de recherche.
Embedding	Vecteur numérique calculé par le modèle pour encoder un texte.
Retriever	Composant qui interroge le vectorstore et retourne les segments pertinents.
Vectorstore	Interface de stockage/recherche des vecteurs; ici LanceDB via LangChain.
LLM	Modèle de langage génératif; servi ici par Ollama.
JWT access	Jeton signé de courte durée envoyé en Bearer.
Refresh token	Secret opaque de longue durée, conservé côté serveur uniquement sous forme hachée.
Guard Angular	Règle de navigation côté client, sans valeur de sécurité suffisante à elle seule.
Dependency FastAPI	Fonction injectée dans une route pour partager authentification, session DB ou autre dépendance.

Fichiers à ouvrir pendant les questions

Point d'entrée : `sys_gestion_doc/api/main.py`. **Route étudiée :** `sys_gestion_doc/api/routes/query.py`. **Pipeline :** `loader.py`, `vectorstore.py`, `rag.py`, `config.py`. **Auth :** `api/auth/router.py`, `dependencies.py`, `security.py`, `ldap_service.py`, `user_service.py`. **Frontend :** `rag-frontend/src/app/app.routes.ts`, `auth.interceptor.ts`, `auth.service.ts`, `shared/services/rag.service.ts`, `chat/chat.component.ts`, `documents/documents.component.ts`. **Déploiement :** `docker-compose.yml`, `k8s/`, `rag-frontend/Dockerfile`.
